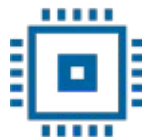




Departamentul Automatică
și Tehnologia Informației



Universitatea
Transilvania
din Brașov
FACULTATEA DE INGINERIE ELECTRICĂ
ȘI ȘTIINȚA CALCULATOARELOR

ROBOT DE URMARIRE A LINIEI

Student(ă):

Jipa Stefan Ciprian

Grupa:

4LF442

Profesor îndrumător:

BRAȘOV, 2025

CUPRINS

INTRODUCERE – OBIECTIVE.....	1
ARHITECTURA.....	2
HARDWARE.....	3
PARCURS ASAMBLARE.....	5
SOFTWARE.....	6
PROBLEME ȘI PROVOCĂRI ÎNTÂMPINATE.....	21
ANSAMBLU FINAL – CONCLUZII.....	23
Poze – prezentare:.....	24
BIBLIOGRAFIE.....	25

INTRODUCERE – OBIECTIVE

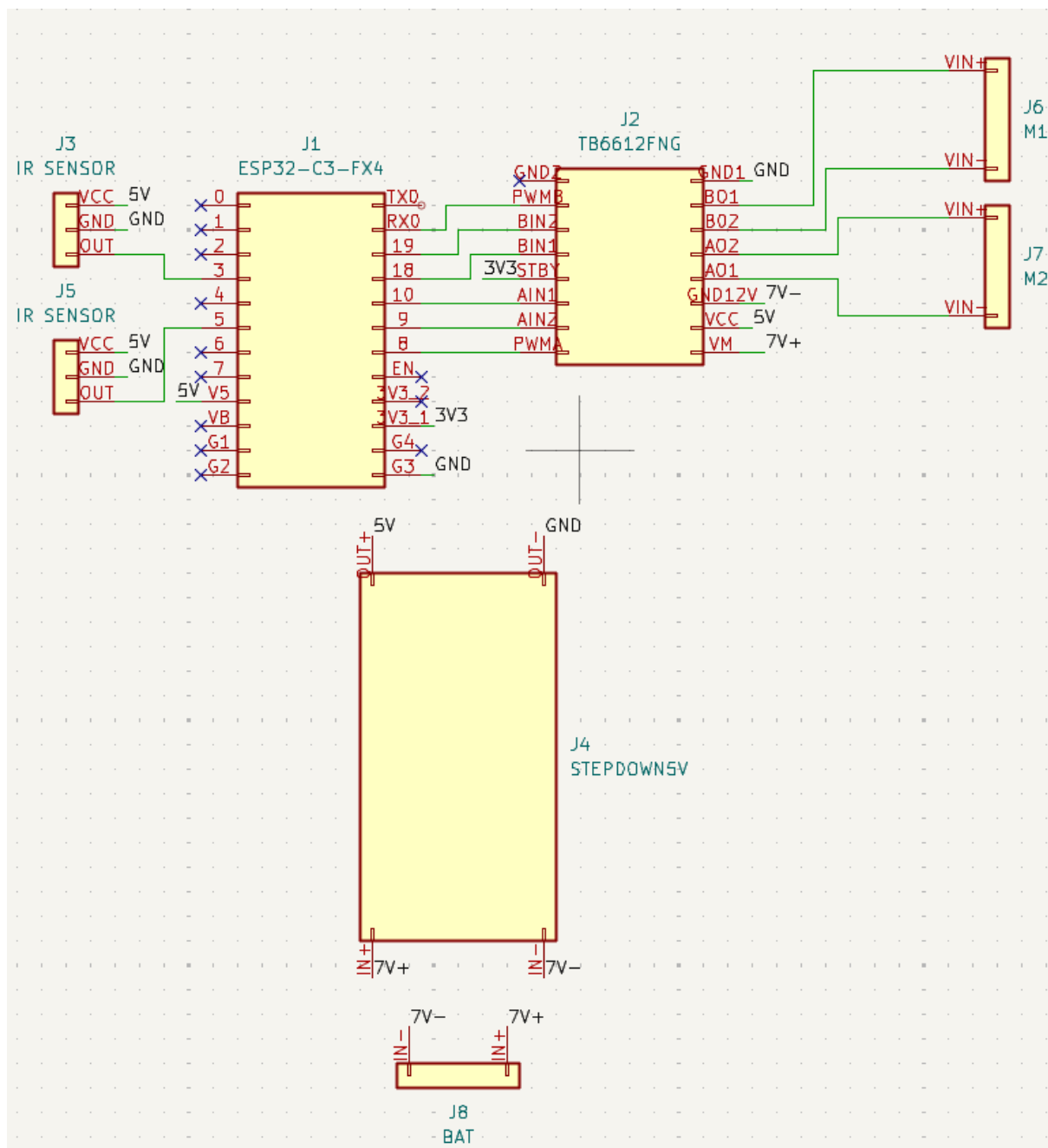
Acest proiect reprezintă primul meu robot real: un line follower clasic, construit integral de la zero. Scopul principal este simplu: robotul să urmeze cât mai corect și cât mai rapid o linie trasă pe podea (de obicei neagră pe fond alb). Prin acest proiect vreau să învăț și să înțeleg în practică următoarele:

- cum se citesc și interpretează senzorii
- cum se ia decizia de direcție
- cum se controlează motoarele cu PWM
- cum se obține stabilitate și viteză rezonabilă
- cum se rezolvă problemele reale care apar la asamblare și testare

Este un exercițiu complet de robotică DIY: electronică, programare, mecanică și debugging – toate la un loc. În continuare voi prezenta pe rând arhitectura generală, componentele hardware, modul de asamblare, codul scris, problemele întâlnite și concluziile finale.

ARHITECTURA

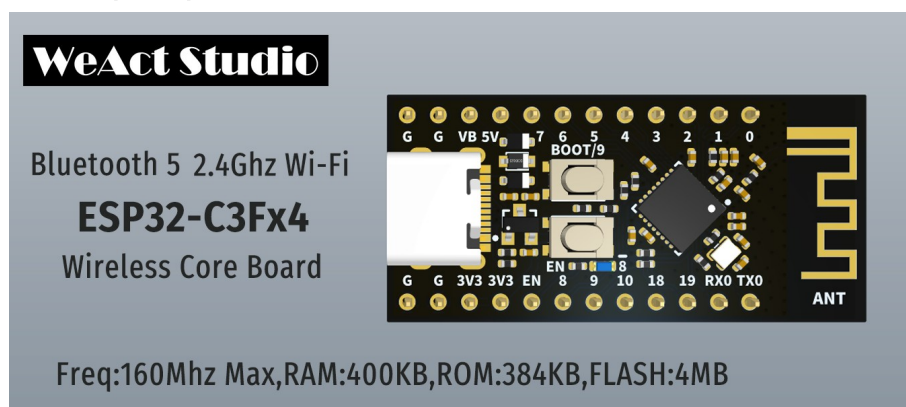
În faza de proiectare inițială, arhitectura robotului a fost gândită simplu și schematic: un microcontroler central care citește direct doi senzori de linie, controlează două motoare prin intermediul unui driver și este alimentat dintr-o sursă unică.



HARDWARE

Pe parcurs, după testele practice și ajustările reale, arhitectura finală a ajuns la următoarea configurație:

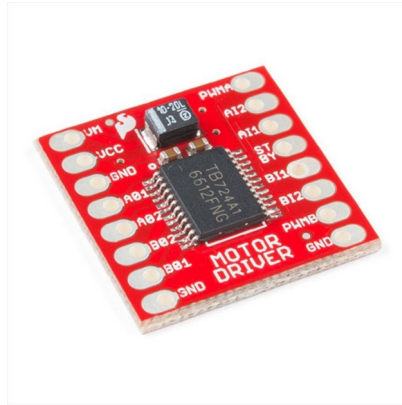
- **Microcontroler principal** WeAct Studio ESP32-C3FH4 (RISC-V, 160 MHz, foarte compact)



- **Senzori** 2 senzori digitali de linie (module albastre ieftine cu ieșire digitală HIGH/LOW)



- **Control motoare** Driver TB6612FNG (PWM + direcție + standby, eficient energetic)



- **Actuatoare** 2 motoare DC cu reductor 50:1 + roți din plastic simple

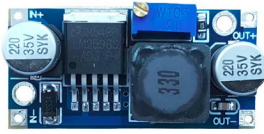


- **Alimentare**

- 2 acumulatori Li-Ion 18650 conectați în serie (~7.4–8.4 V)



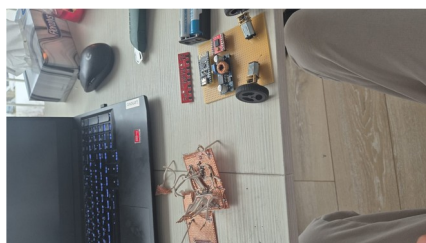
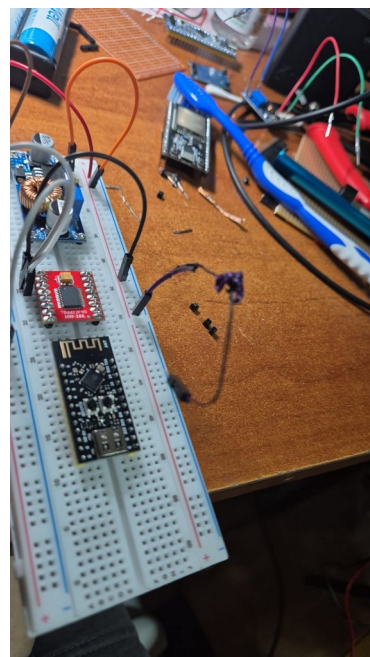
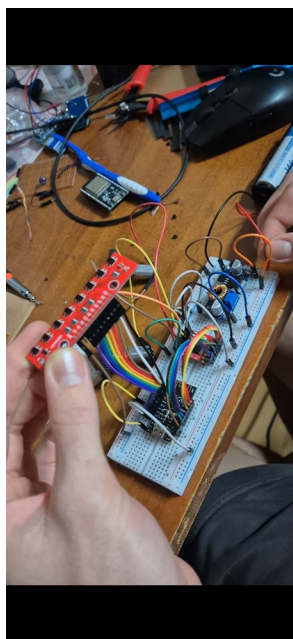
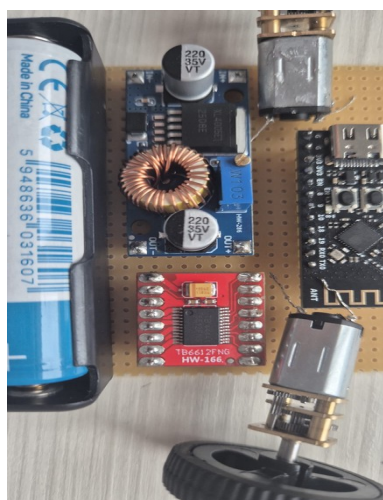
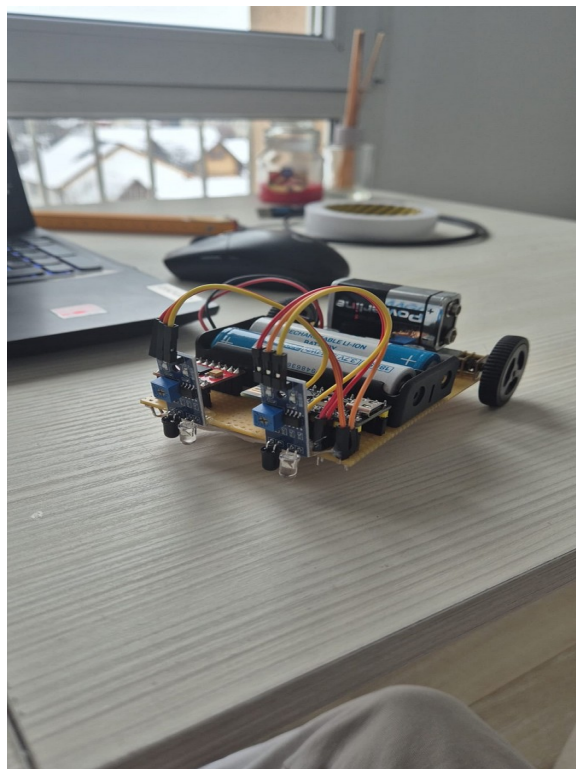
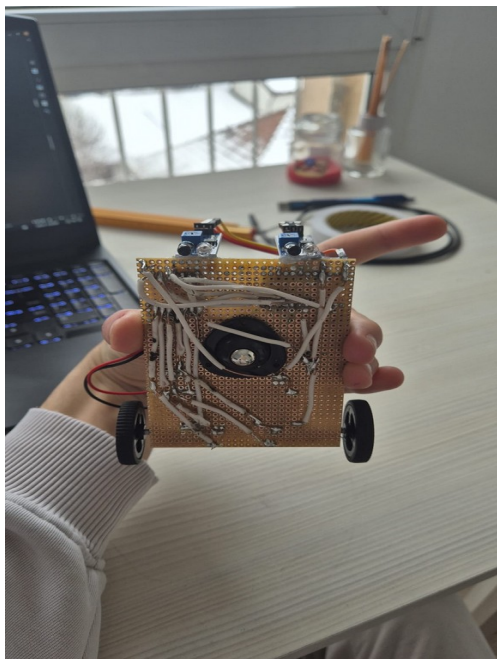
- Modul step-down (buck) la 5 V stabil pentru ESP32, driver și senzori



- Switch mecanic simplu pentru pornire/oprire totală
- **Conexiuni principale**
 - Senzorii → direct la 2 pini GPIO digitali ai ESP32
 - Driverul TB6612FNG → 6 pini GPIO (2× PWM + 4× direcție)
 - Alimentare logică și senzori → 5 V de la step-down
 - Motoare → direct de la ieșirile driverului (alimentate cu ~7.4 V)

Această arhitectură finală este cea implementată, testată și folosită în toate măsurătorile și demonstrațiile ulterioare.

PARCURS ASAMBLARE



SOFTWARE

Instrumente software folosite (cât se poate de simplu și direct):

- **Arduino IDE** (singurul mediu folosit de la început până la final)
- Framework **Arduino pentru ESP32**

Pentru a putea adăuga și folosi placa WeAct ESP32-C3 în Arduino IDE, am adăugat acest URL în preferințe → Additional Boards Manager URLs:

https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_dev_index.json

După aceea:

- Tools → Board → Boards Manager
- căutat „esp32”
- instalat pachetul Espressif Systems (ultima versiune disponibilă)

Fișierul principal al proiectului: **linefollower.ino**:

```
#include <SparkFun_TB6612.h>

// Motor pins
#define AIN1 18
#define AIN2 19
#define PWMA 20

#define BIN1 10
#define BIN2 9
#define PWMB 8

#define STBY 30

constexpr int OFFSET_A = 1;
constexpr int OFFSET_B = 1;
```

```

Motor motorStg(AIN1, AIN2, PWMA, OFFSET_A, STBY); // left motor
Motor motorDrp(BIN1, BIN2, PWMB, OFFSET_B, STBY); // right motor

// Sensors
constexpr uint8_t SENSOR_STANGA = 2;
constexpr uint8_t SENSOR_DREAPTA = 5;

// Speeds (0–255)
constexpr int8_t V_CRUISE = 80;
constexpr int8_t V_SEARCH = 55;
constexpr int8_t V_TURN = 0;
constexpr int8_t V_CORRECT = 25;

unsigned long lastCorrection = millis(); // last correction time
constexpr unsigned long TIMEOUT_PIERDUT = 250; // lost timeout in ms

void setup() {
  Serial.begin(115200);
  delay(800); // boot stabilization

  pinMode(SENSOR_STANGA, INPUT);
  pinMode(SENSOR_DREAPTA, INPUT);

  Serial.println("Line follower ESP32-C3 started");
}

void loop() {
  bool L = digitalRead(SENSOR_STANGA); // 1 = black line
  bool R = digitalRead(SENSOR_DREAPTA);

```

```

int8_t vitezaStg, vitezaDrp;

if (!L && !R) { // 00 both white
if (millis() - lastCorrection > TIMEOUT_PIERDUT) {
// line lost, search
vitezaStg = V_SEARCH;
vitezaDrp = -V_SEARCH;
} else {
// go straight
vitezaStg = V_CRUISE;
vitezaDrp = V_CRUISE;
}
} else if (L && !R) { // 10 line on left
vitezaStg = V_CORRECT;
vitezaDrp = V_TURN;
lastCorrection = millis(); // update timer
} else if (!L && R) { // 01 line on right
vitezaStg = V_TURN;
vitezaDrp = V_CORRECT;
lastCorrection = millis(); // update timer
} else { // 11 both black
vitezaStg = 0; // short stop
vitezaDrp = 0;
delay(50);
}

motorStg.drive(vitezaStg);
motorDrp.drive(vitezaDrp);

// Debug only
// static uint32_t t = 0;

```

```
// if (millis() - t > 120) {  
// Serial.printf("%d%d %4d %4d\n", L, R, vitezaStg, vitezaDrp);  
// t = millis();  
// }  
}
```

Principalele caracteristici ale algoritmului:

- Folosește doar **2 senzori digitali** (stânga și dreapta)
- Senzorii sunt de tip **HIGH = linie neagră, LOW = alb**
- Controlul se face prin **4 stări simple** (00, 01, 10, 11)
- Corecție diferențială: reduce viteza unei roți și oprește/opune cealaltă pentru viraj
- Când pierde linia (>250 ms fără corecție) → începe să caute rotindu-se pe loc
- Când ambii senzori pe negru → oprește scurt (evită să intre în intersecții sau capete de linie)

Constanta importante tunate manual:

- V_CRUISE = 80 → viteza normală pe linie dreaptă
- V_SEARCH = 55 → viteza de căutare (rotire pe loc)
- V_CORRECT = 25 → viteza redusă la corecție
- V_TURN = 0 → roata din interior stă pe loc la viraj
- TIMEOUT_PIERDUT = 250 ms

Cum funcționează în loop():

- Stare senzori → Acțiune
- 00 (ambii albi) → dacă recent a văzut linie → merg drept
dacă a trecut timeout → caută rotindu-se
- 10 (doar stânga) → viraj stânga (stânga încet, dreapta normal)
- 01 (doar dreapta) → viraj dreapta (dreapta încet, stânga normal)
- 11 (ambii negri) → stop scurt 50 ms

Biblioteci folosite:

- SparkFun_TB6612.h (pentru driverul TB6612FNG – simplifică mult controlul motoarelor)

Observații utile:

- Codul este foarte curat și scurt (~70 linii funcționale fără comentarii)
- Nu există PID → doar logică de stare (funcționează surprinzător de bine după tuning)
- Debug-ul prin Serial este comentat (poți activa ușor când testezi)

PROBLEME ȘI PROVOCĂRI ÎNTÂMPINATE

Fiind primul proiect serios, aproape totul a mers cu probleme, mai ales din cauza încercărilor de a face robotul mic și puternic de la început.

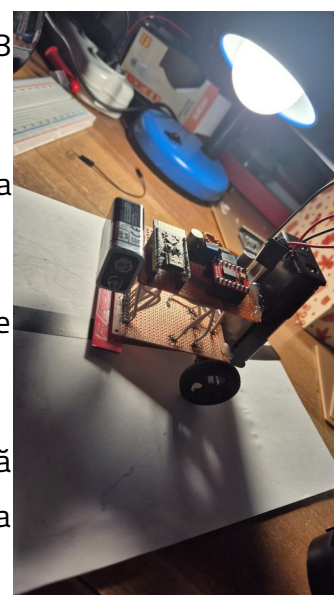
1. Primele încercări de design compact – eșec total

Inițial voiam un robot cât mai mic, așa că am luat un array de 8 senzori IR + 2 plăci de prototipare 5×7 cm puse sandviș.

Problema majoră: centrul de greutate era nasol, robotul voia mereu să se dea pe spate.

Plus că aveam un step-up de la 7 V la 12 V pentru „putere maximă” la motoare – dar a complicat totul și consuma aiurea.

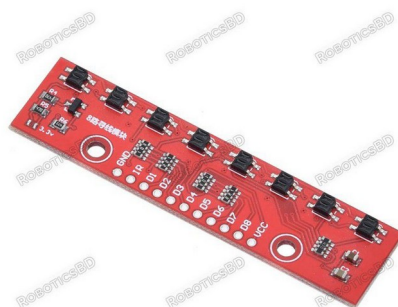
După multe teste, am renunțat la ambele: am trecut pe o singură placă mai mare 7×10 cm și am lăsat motoarele direct la 7,4 V de la bateriile 18650.



2. Senzorul de 8 canale – ars și înlocuit

În timp ce tot sudam/desusdam array-ul de 8 senzori, l-am ars complet (prea multe suduri, prea multă căldură).

A trebuit să renunț la el și să-l înlocuiesc cu doar 2 senzori digitali simpli (albastru ieftin) deoarece era prea tarziu sa mai comand unul nou.



Asta a schimbat totul: cu 2 senzori nu mai poți face algoritmi avansați, virajele sunt mai brute, pierde linia mai ușor în curbe. Dar măcar merge.



3. Suduri foarte grele și cronofage

Am stat 3 zile bune doar cu lipit/deslipit.

Pe primul model (sandviș) era extrem de complicat să sudez între cele două plăci pe înălțime.

Pe al doilea model a fost ceva mai ok, dar tot a rămas multă muncă meticuloasă și stresantă.

4. Centru de greutate prost și la modelul final

Chiar și pe placa mai mare, robotul avea aderență slabă la roți.

Am fost nevoit să lipesc o baterie pătrată de 9 V în spate doar ca contragreutate, ca să nu patineze.

5. Aproape am prăjit driverul TB6612FNG

L-am pus invers prima dată → 7 V și 5 V au intrat în pinii logici.

Noroc că am observat repede, m-am ales doar cu o mică arsătură pe deget. Driverul a supraviețuit.

6. Probleme cu plăcile testate

Am început cu un NodeMCU-S3 (sau ceva asemănător), dar avea o grămadă de erori cu senzorii și mă enerva portul microUSB.

Am trecut la WeAct ESP32-C3 – mult mai mică, compactă și frumoasă, dar dacă știam de la început că rămân la 2 senzori, aș fi ales altceva.

A ocupat toți pinii buni, inclusiv RX0/TX0 (folosiți pentru Serial), așa că debug-ul prin Serial a devenit imposibil fără să fac secvența de boot manuală (boot → EN → -EN → -boot) de fiecare dată când voiam upload.

7. Comportament haotic până la final

Până în ultimele 3 teste robotul nu semăna deloc cu un line follower:

- se învârtea în cerc nonstop
- căuta ceva imaginar
- se dădea peste cap

Abia după ce am simplificat mult codul (logică de stare foarte basică, fără complicații) a început să meargă cât de cât decent.

ANSAMBLU FINAL – CONCLUZII

Mă oftic puțin că nu am putut face robotul mai performant – arsura senzorului de 8 canale a fost lovitura decisivă.

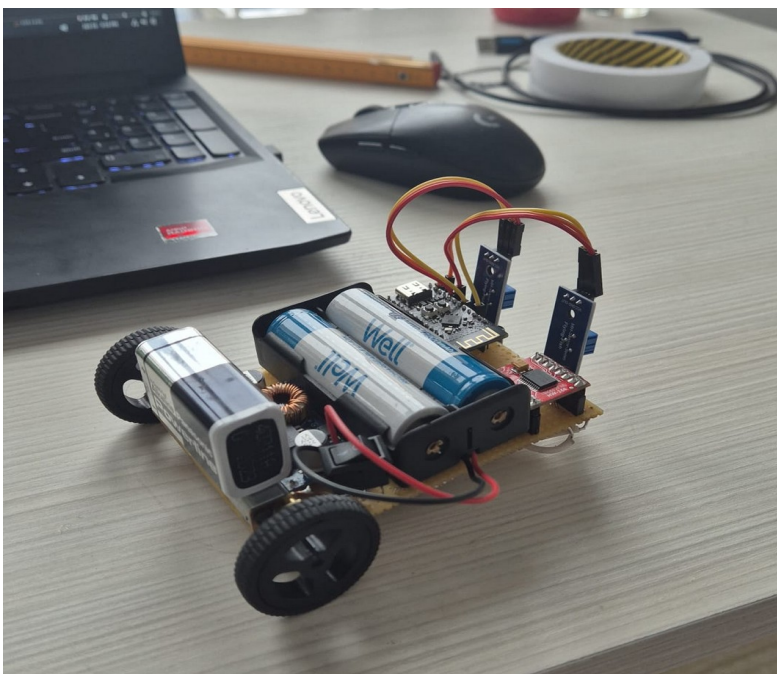
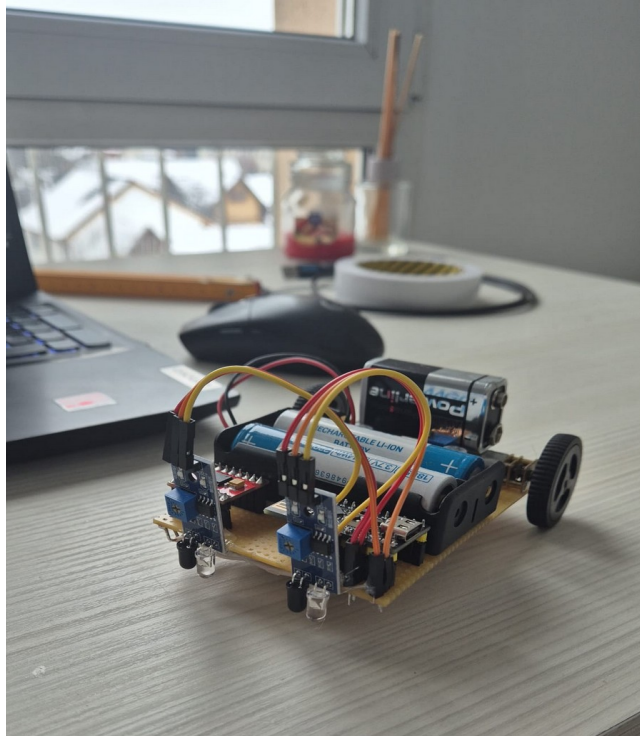
Dar am învățat enorm: simplitatea bate perfecțiunea la început, calitatea componentelor și cablurilor contează, iar tuning-ul manual cere răbdare.

În final, robotul face ce trebuie: urmează linia decent cu viteză rezonabilă și fără să se dea peste cap.

Chiar dacă nu e perfect, sunt mulțumit că am reușit să-l fac funcțional de la zero, cu ce am avut la îndemână.

Proiectul ăsta mi-a arătat clar că răbdarea și simplificarea pas cu pas sunt cele mai importante la robotică DIY.

POZE – PREZENTARE:



BIBLIOGRAFIE

<https://github.com/WeActStudio/WeActStudio.ESP32C3CoreBoard>

<https://learn.sparkfun.com/tutorials/tb6612fng-hookup-guide/all>

<https://projecthub.arduino.cc/abhilashpatel121/using-ir-sensor-tcrt-5000-with-arduino-60cc92>

<https://mischianti.org/weact-studio-esp32-c3-core-high-resolution-pinout-and-specs/>

<https://www.youtube.com/watch?v=PP4fvBVe3rI>

<https://www.youtube.com/watch?v=cMZDbDiHYzQ&list=PLHWHUGHuOZnW0xLBJh2SKwC17P59EQN2s&index=3>

<https://grok.com>